pfi

Python for Fixed Income

COURSE 8 · IMPLEMENTING AGENTIC AI

MCP: Connecting Data Sources

WEEK 2 · DAY 3

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Today: MCP, connecting data sources

Day 6: tools in the notebook

`tool_schema`	describes each function
`run_tools`	runs it in this process
`agent_loop`	holds it together

Today

1. What MCP is
2. course_server
3. The in-process client
4. mcp_tool_schemas
5. One question, local and through MCP
6. The same server over stdio
7. Version 2's names
8. What MCP adds, and not

Every model response in the notebook is a saved response, replayed. A live run can differ.

The rules for agents

What an agent produces: text, every number from a tool, wherever the tool runs

Allowed names	Never called
the agent returned	the agent's opinion or view
the saved run	what the agent believes
the brief states	the agent's advice

On the Treasury curve: a level, a change, a slope from the file, never a view on rates

Allowed names	Never called
the 10 Yr par yield in the file	a forecast or an outlook
the change in bp between two dates	a signal
the slope in bp	a view on rates

A server, a client, and a protocol

Model

receives a tool list,

returns tool requests

Client: your loop

lists the tools, sends

each request on

Server

lists its tools, runs

them, returns results

tool list and requests (JSON, as on day 2)

MCP: list_tools, call_tool (the model never sees it)

changes

where the tool runs

unchanged

what the model is sent

mcp 2.3.0

MCPServer, not FastMCP

The agent loop stays on the client side, as on day 2: the model is sent a tool list and returns tool requests.

course_server: your own tools, served

```
market = agentkit.market_tools(curve, spreads)   # {name: function}, as run_tools takes
server = agentkit.course_server(market)
# server: course-data; class: MCPServer
```

docstring

becomes the description

type hints

become the input schema

WARNING

so stderr stays empty

The docstring is still the prompt. It now travels through a server.

The in-process client: list_tools and call_tool

```
treasury_yield     required ['tenor', 'as_of']
curve_on           required ['as_of']
curve_change_bp    required ['tenor', 'start', 'end']
slope_bp           required ['short', 'long', 'as_of']
synthetic_spread   required ['kind', 'as_of']
```

a date in the file: is_error False	`{ "tenor": "10 Yr", "as_of": "2026-10-02", "yield_pct": 5.28 }`
the date left out: is_error True	`Error executing tool treasury_yield: 1 validation error for treasury_yieldA...`
a date the file lacks: is_error False	`{ "error": "no row for that date in the curve file: 2026-10-03 10 Yr" }`

Two kinds of error: arguments the schema refuses (the server's), an answer the data lacks (the tool's).

mcp_tool_schemas: the listing in the raw loop's format

treasury_yield as MCP lists it

```
{"type": "object", "properties": {"tenor": {"title": "Tenor", "type": "string"}, "as_of": {"title": "As
Of", "type": "string"}}, "required": ["tenor", "as_of"], "title": "treasury_yieldArguments"}
```

the same tool, converted: titles gone, Args: descriptions added, required kept

```
{"type": "object", "properties": {"tenor": {"type": "string", "description": "the tenor as the file labe
ls it: 1 Mo, 3 Mo, 6 Mo, 1 Yr, 2 Yr, 3 Yr, 5 Yr, 7 Yr, 10 Yr, 20 Yr, 30 Yr."}, "as_of": {"type": "string
", "description": "the date as YYYY-MM-DD, for example 2026-10-02."}}, "required": ["tenor", "as_of"], "
additionalProperties": false}
```

Equal to tool_schema's for all five tools: the model can be sent the same list, byte for byte.

One question, local and through MCP

```
run_local = ask(QUESTIONS["change"], market)                    # agent_loop, tools in this process
run_mcp = await mcp_loop(session, QUESTIONS["change"])          # tools on the server: one await
```

Request keys of the two saved runs (first 12 characters)

```
local  '2fa97943a81f', '2b8929ae57c8', '726ee6b407b9'

MCP    '2fa97943a81f', '2b8929ae57c8', '726ee6b407b9'
```

same request keys: True same answer: True figures 2 of 2 passed

Model gpt-4.1-mini-2025-04-14, temperature 0, seed 42; 3 requests, 3,432 prompt and 263 completion tokens; saved run recorded 2026-10-11; replayed (no key loaded).

The model's side of the run cannot tell where the tools ran.

The same server over stdio: a separate process

This notebook: the client

```
stdio_client, ClientSession, mcp_loop
```

course_server.py: its own process

```
server.run("stdio")
```

stdin and stdout: the protocol stderr: a log file

2025-11-25

protocol version

True

same request keys

0 bytes

the server's stderr

A tool that raised would write a traceback to stderr. The course's tools return an error result instead.

Version 2's names: an import that now fails

```
from mcp.server.fastmcp import FastMCP      # the version 1 import most tutorials give
```

```
No module named 'mcp.server.fastmcp'
This is mcp 2.x, where FastMCP was renamed to MCPServer (from mcp.server.mcpserver import MCPServ...
see the migration guide at https://py.sdk.modelcontextprotocol.io/v2/migration/#fastmcp-renamed-t...
pin 'mcp<2' to keep running v1 code.
```

```
from mcp.server.mcpserver import MCPServer   # version 2: what the message says
```

Read a migration message last part first. The course keeps version 2; it does not pin mcp<2.

What MCP adds, and what it does not

	Day 6	Through MCP
Tool list sent	tool_schema	mcp_tool_schemas: the same
Where a tool runs	this kernel	the server's process
Wrong argument shape	"wrong arguments"	is_error, pydantic's text
A date the file lacks	the tool's error	the same, passed through
Request bodies	agent_loop's	the same keys
A figure	check_figures, yours	check_figures, yours

```
local  figures 2, passed 2; values [27.0, 45.0]
MCP    figures 2, passed 2; values [27.0, 45.0]
stdio  figures 2, passed 2; values [27.0, 45.0]
```

MCP carries tools and checks their arguments' shape. It checks no number.

Excel and Git

Excel side by side (tested in Excel)

`=(INDEX(B2:O2941,MATCH(Q2,A2:A2941,0),12)-INDEX(B2:O2941,MATCH(Q1,A2:A2941,0),12))*100`	27
`=(INDEX(B2:O2941,MATCH(Q2,A2:A2941,0),12)-INDEX(B2:O2941,MATCH(Q2,A2:A2941,0),8))*100`	45
`=INDEX(B2:O2941,MATCH(Q3,A2:A2941,0),12)`	#N/A

Excel against the tools on the same file: 3 of 3 rows the same. Excel has no MCP client.

```
agentkit.py | 79 ++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
course_server.py | 19 +++++++++++++++
test_agentkit.py | 47 ++++++++++++++++++++++++++++++++++
```

course_server.py committed with the day's code. 43 passed.

The AI check

The docstring is the prompt. The agent code that comes back runs and looks right.

1

Docstring

Properties and the required list, exactly as the server lists them.

→

2

Code back

Written for the course in the style of an AI answer. Read it.

→

3

Test first

Every tool's required list must survive. Run it: it fails.

→

4

Fix

Copy required with the properties. 5 of 5 kept.

Today: the converter copies the properties and drops required, so every argument looks optional to the model.

Code that runs is not the same as code that is right.

What you can do now

- A server lists and runs tools; a client connects
- course_server from your own functions
- mcp_tool_schemas, required kept
- Through MCP: the same request keys
- stdio as its own process, stderr in a file; MCP checks no number

Week 2, Day 4 The framework: the loop in the Agents SDK